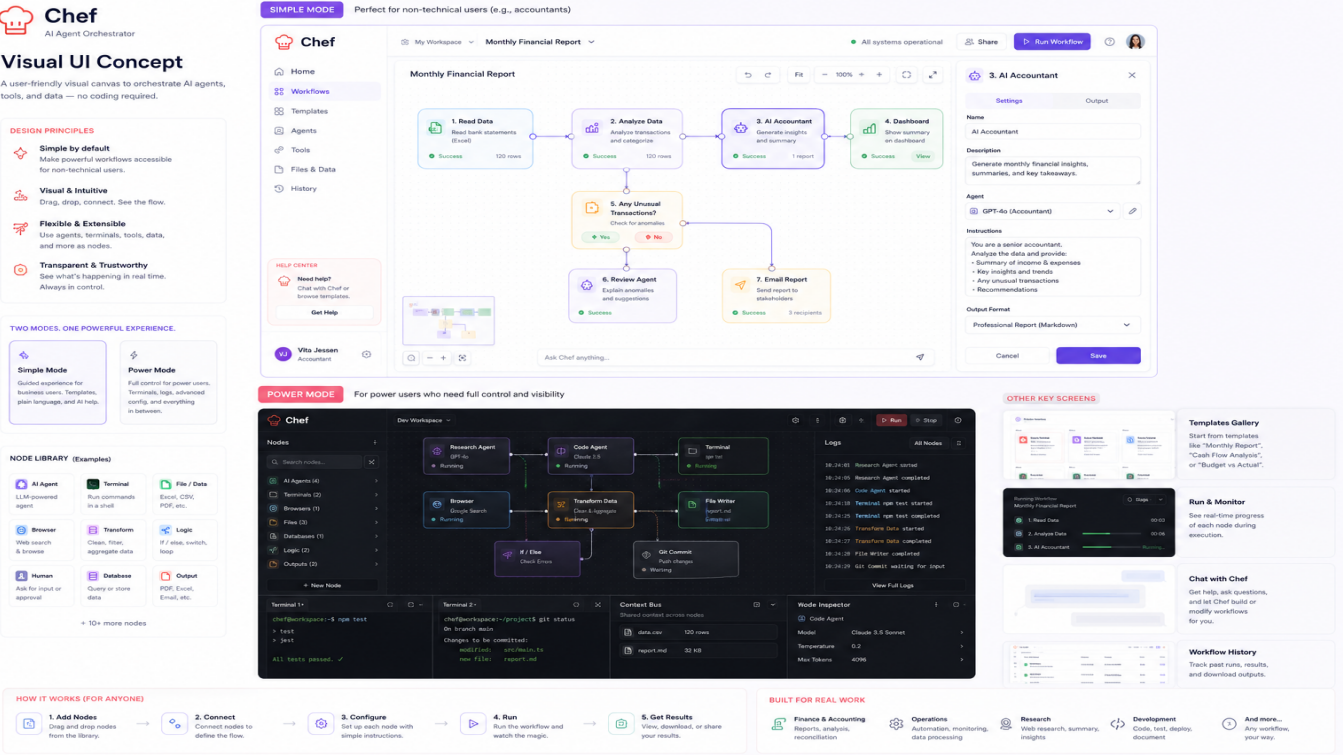


Chef — UI / UX Visual Specification

A visual AI workbench for everyone: accountants, operators, researchers, and developers.



Reference mockup: Simple Mode + Power Mode + supporting screens.

1. Product Principle

Chef is not a coding canvas. It is a visual workspace where people compose useful workflows from reusable nodes. The same runtime can power a monthly financial report or a multi-agent software-development pipeline.

Node	Purpose	Example
AI Agent	LLM-powered reasoning or execution	AI Accountant / Code Agent
Terminal	Real shell / command execution	npm test / Python script
File / Data	Read structured or unstructured data	Excel / CSV / PDF
Browser	Web research and interaction	Research competitor prices
Transform	Clean, map, aggregate data	Group transactions
Logic	Branch, loop, condition	If anomaly > threshold
Human	Approval or input checkpoint	Approve report
Output	Produce or deliver results	PDF / Excel / Email

2. Two Modes, One Runtime

Simple Mode: default for non-technical users. Friendly labels, templates, guided setup, plain-language configuration, Chat with Chef, and minimal runtime details.

Power Mode: for developers and advanced users. Full node graph, terminals, browser surfaces, logs, context bus, agent configuration, execution state, and low-level controls. Users can switch modes without rebuilding the workflow.

3. Core Canvas Layout

Left: navigation + searchable Node Library. **Center:** infinite canvas with connected nodes. **Right:** contextual Inspector for the selected node. **Bottom:** execution console / results / Chat with Chef. The canvas must remain the visual center of gravity.

4. UX Rules

- Drag → connect → configure → run. A first-time user should understand this without documentation.
- Every node has a human-readable name, icon, status, inputs, outputs, and a short description.
- Use progressive disclosure: simple controls first; advanced controls behind expandable sections.
- Never expose raw model/runtime terminology unless the user enters Power Mode.
- Execution must be observable: users can see which node is running, completed, failed, or waiting.
- Chat with Chef should be able to explain, build, modify, and troubleshoot workflows.
- Templates are first-class: start from a useful workflow rather than an empty canvas.
- Human approval should be a native node, not a special-case dialog.

5. Example: Accountant Workflow

Read Excel → Analyze Transactions → AI Accountant → Detect Anomalies → Review → Generate PDF → Email

The accountant should be able to import an Excel file, select a template, answer a few questions, run the workflow, review the generated report, and download/share it without knowing what an LLM, terminal, agent harness, or context bus is.

6. Example: Developer Workflow

Orchestrator → Research Agent → Code Agent → Terminal → Browser → Test Agent → Review Agent → Git

Power Mode exposes the same underlying workflow as a development graph. Terminals are real interactive surfaces, agents can communicate through connected inputs/outputs and shared context, and execution/log state is visible.

7. Visual Direction

Modern desktop SaaS rather than an IDE. Clean whitespace, rounded cards, subtle borders, clear typography, soft color-coded node categories, restrained gradients, and strong status indicators. Simple Mode should feel approachable; Power Mode can become denser and darker without changing the underlying mental model.

Implementation target: build the UI as a real product system, not a static mock. Every major element should map to a real Chef capability or an intentionally extensible interface.